

EXOCENTRIC NAVIGATION SYSTEM FOR AUTONOMOUS ROBOTS

Project report submitted in partial fulfillment of the requirement for the degree of

Bachelor of Technology
in
Electrical and Computer Engineering

Submitted by

Aravind Abburi (2210110826)

And

Divyansh Dhani (2210110709)

Under Supervision of

Dr. Kumar Saurav
Department of Electrical Engineering



Department of Electrical Engineering
School of Engineering
Shiv Nadar Institution of Eminence
(April 2026)

CANDIDATE DECLARATION

We hereby declare that the thesis entitled “**Exocentric Navigation System for Autonomous Robots**” submitted for the B. Tech. degree program. This thesis has been written in our own words. We have adequately cited and referenced the original sources.

(Signature)
Aravind Abburi
(2210110826)

(Signature)
Divyansh Dhani
(2210110709)

CERTIFICATE

It is certified that the work contained in the project report titled “**Exocentric Navigation System for Autonomous Robots**” by **Aravind Abburi (2210110826)** and **Divyansh Dhani (2210110709)** has been carried out under our supervision and that this work has not been submitted elsewhere for a degree.

(Signature)

Dr. Kumar Saurav

Dept. of Electrical Engineering

School of Engineering

Shiv Nadar Institution of Eminence

Date: --- / --- / 2026

ABSTRACT

This project presents a real-time autonomous robot navigation system that integrates computer vision, path planning, and wireless communication. The system uses a camera to monitor the environment, where the robot is detected and tracked using motion detection, visual tracking, and filtering techniques. A Kalman filter is applied to improve the accuracy and stability of robot position estimation.

Obstacle detection is performed using a deep learning model based on YOLO, which identifies obstacles in the environment and provides their spatial locations. The detected obstacles are mapped onto a grid-based representation, enabling efficient navigation.

A grid-based A* path planning algorithm is implemented to compute the shortest and safest path from the robot's current position to a user-defined target. Additional improvements such as obstacle padding, soft constraints, and path memory are incorporated to ensure smooth and collision-free navigation.

The system employs pulse-based control for robot movement, where short-duration commands are sent to the robot instead of continuous signals. These pulses are generated based on real-time robot detection and orientation, allowing precise and controlled motion. This approach reduces overshooting and improves stability during navigation.

Wireless communication is established using the ESP8266 module with UDP protocol. The ESP8266 forwards the received commands to an Arduino microcontroller, which controls the robot motors through a motor driver.

The overall system operates in real-time and demonstrates reliable performance in robot detection, obstacle avoidance, and navigation. This project highlights the integration of artificial intelligence, computer vision, and embedded systems to develop an intelligent and efficient robotic system.

Contents

1	Introduction	9
1.1	Components:	9
1.2	External USB Camera:	10
1.3	Autonomous robot :	10
1.4	Arduino Uno:	11
1.5	ESP8266 Wifi module:	12
2	Literature Survey	13
2.1	Reviewed Papers and Identified Limitations	13
2.1.1	Tselishchev (2024) – Autonomous Navigation Using Overhead Camera	13
2.1.2	Pribanic et al. – Global Vision-Based Multi-Robot Tracking	13
2.1.3	Redmon et al. – YOLO (You Only Look Once)	14
2.1.4	Bradski – OpenCV Library	14
2.1.5	Hart, Nilsson, Raphael – A* Algorithm	15
2.1.6	Ultralytics – YOLOv8 Documentation	15
2.1.7	Roboflow – Computer Vision Platform	15
2.2	Summary of Research Gaps	16
3	Work Done Before Mid-Sem	17
3.1	Object Detection	17
3.1.1	Theory	17
3.2	Robot Image Data Collection Using OpenCV	18
3.2.1	YOLO Algorithm	18
3.3	Dataset Preparation and Model Training	18
3.4	Robot Detection and Position Estimation	19
3.5	System Overview	19
3.6	Sample Object Detection Code	20
4	Work Done After Mid-Sem	21
4.1	Wireless Communication using ESP8266	21
4.2	Arduino-Based Robot Control	21
4.3	Improved System Integration	22
4.4	System Design and Working	22
4.4.1	Overall Idea	22
4.4.2	Network and Model Setup	22
4.4.3	Grid System (Environment Mapping)	23

4.4.4	Calibration (Homography)	23
4.5	Transition to Pulse-Based Robot Detection	23
4.6	Robot Detection and Tracking	24
4.6.1	Motion Detection Scoring	24
4.6.2	Continuous Tracking	24
4.6.3	Kalman Filter Smoothing	24
4.7	Robot Direction Detection	24
4.8	Target Selection	25
4.9	A* Path Planning	25
4.10	Obstacle Detection using YOLO	25
4.11	Navigation Logic	26
4.12	Lookahead Point	26
4.13	State Machine	26
5	Conclusion	27
6	Future Work	28
6.1	PID-Based Motion Control	28
6.2	Multiple Camera System	28
6.3	SLAM (Simultaneous Localization and Mapping)	29
6.4	Sensor Fusion	29
6.5	Dynamic Obstacle Handling	29
6.6	Improved Communication Protocols	29
	References	30

List of Figures

1.1	USB camera	10
1.2	Autonomous robot	11
1.3	Arduino Uno	11
1.4	esp8266	12
3.1	Robot Detection	17
4.1	Robot Direction	24

List of Tables

1.1	Hardware Architecture and Roles	9
2.1	Comparison of Object Detection Frameworks	16
2.2	System Parameters and Targets	16

Chapter 1

Introduction

Autonomous robot navigation is an important area of research in robotics, where robots must move from one location to another while avoiding obstacles and reaching a target destination. Most traditional navigation systems rely on sensors and cameras mounted directly on the robot. These systems operate from a first-person perspective, meaning the robot perceives the environment only from its own viewpoint. Although effective in many cases, such approaches can face limitations such as restricted field of view, sensor noise, and difficulty in coordinating multiple robots in the same workspace.

To overcome these limitations, this project proposes an exocentric navigation system, where the robot is observed from an external camera placed outside the robot. This camera provides a global, third-person view of the entire environment, allowing the system to monitor multiple robots and obstacles simultaneously. By using a centralized view of the workspace, the navigation system can make more informed decisions and improve overall movement planning and safety.

1.1 Components:

Table 1.1: Hardware Architecture and Roles

Component	System Role	Primary Function
Central PC / Laptop	God Brain	Runs YOLOv8 object detection, computes A* path planning math, and sends UDP Wi-Fi commands.
External Webcam	Eyes	Captures a global, exocentric view of the workspace and feeds frames to the PC.
ESP32 / ESP8266	Ears (Receiver)	Connects to the local network, receives UDP commands from the PC, and transmits them via Serial (TX/RX).
Arduino Uno	Muscle (Controller)	Interprets Serial commands from the ESP and directly drives the DC motors via a motor driver.

The proposed system is built using several hardware components, including a central user laptop, an Arduino-based autonomous robot, an ESP8266 Wi-Fi module, and a USB camera. The USB camera is connected to the laptop and is used to capture the workspace, including the robot and obstacles present in the environment. The captured video feed is processed on the laptop using computer vision and machine learning algorithms to detect and track the robot and obstacles. The autonomous robot is controlled using an Arduino microcontroller, which manages the robot's movements such as forward, backward, left, and right. The ESP8266 Wi-Fi module enables wireless communication between the laptop and the robot. Through this communication, movement commands generated by the detection and navigation system are sent to the robot in real time, allowing it to navigate safely while avoiding obstacles.

1.2 External USB Camera:



Figure 1.1: USB camera

The external USB camera is an important component of the proposed exocentric navigation system. It is used to capture real-time images and video of the entire workspace from a fixed position outside the robot. Unlike traditional robot navigation systems that use cameras mounted on the robot, this system uses an external camera to provide a global third-person view of the environment. This allows the system to observe multiple robots, obstacles, and the overall layout of the workspace simultaneously.

1.3 Autonomous robot :

In this project, the autonomous robot is designed to move within a defined workspace while receiving navigation commands from the central processing system. The robot is built using the Arduino Uno, which acts as the main controller for the robot's movement.

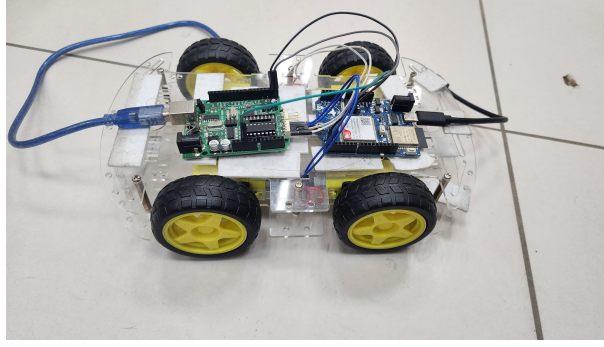


Figure 1.2: Autonomous robot

The Arduino controls the motors of the robot and executes the commands sent by the external system.

1.4 Arduino Uno:

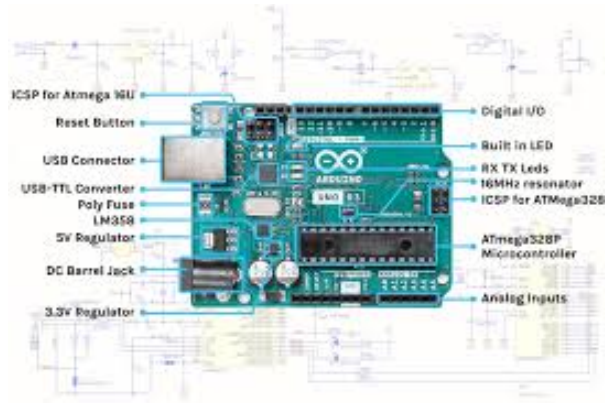


Figure 1.3: Arduino Uno

In this project, the robot is controlled using the Arduino Uno, which acts as the main controller for the autonomous robot. The Arduino Uno is based on the ATmega328P microcontroller and operates at a clock speed of 16 MHz. It includes 14 digital input/output pins, 6 analog input pins, USB connectivity, and power supply pins, making it suitable for various embedded applications.

The Arduino board is responsible for controlling the robot's DC motors through a motor driver module. It receives navigation commands from the ESP8266, which communicates wirelessly with the central PC / laptop. Based on the received commands, the Arduino controls the movement of the robot, such as moving forward, backward, turning left, or turning right.



Figure 1.4: esp8266

1.5 ESP8266 Wifi module:

In this project, the ESP8266 is used to establish wireless communication between the central PC / laptop and the robot controlled by the Arduino Uno. The laptop processes the camera data, determines the robot's navigation path, and sends movement commands through Wi-Fi using the ESP8266 module. The Arduino receives these commands and controls the robot's motors accordingly.

Chapter 2

Literature Survey

2.1 Reviewed Papers and Identified Limitations

A detailed review of key research papers and technical resources was conducted to understand existing approaches in object detection, computer vision, and autonomous navigation. The following summarizes the contributions, limitations, and how each work is utilized in our project.

2.1.1 Tselishchev (2024) – Autonomous Navigation Using Overhead Camera

This work presents a mobile robot navigation system based on an overhead (exocentric) camera. The system captures a global view of the environment to determine robot position and obstacles.

Limitations and Challenges:

- High dependency on controlled lighting
- Processing latency in real-time applications
- Requires precise camera calibration

Use in Our Project:

- Inspired the use of **exocentric vision** using an external USB camera
- Helps in achieving a global view of robot and obstacles
- Forms the base for coordinate extraction and navigation

2.1.2 Pribanic et al. – Global Vision-Based Multi-Robot Tracking

This paper focuses on tracking multiple robots using global vision with high precision.

Limitations and Challenges:

- High computational complexity

- Sensitive to occlusion
- Requires advanced filtering algorithms

Use in Our Project:

- Provides concept of **global tracking**
- Instead of complex filtering, we use **YOLOv8 for detection**
- Simplifies tracking process using bounding boxes

2.1.3 Redmon et al. – YOLO (You Only Look Once)

This paper introduces YOLO, a deep learning-based real-time object detection model.

Limitations and Challenges:

- Requires large training datasets
- Needs GPU for better performance
- Lower accuracy for small objects (early versions)

Use in Our Project:

- Forms the foundation for **real-time object detection**
- Enables detection of robot and obstacles in a single frame
- Improves speed compared to traditional methods

2.1.4 Bradski – OpenCV Library

OpenCV provides tools for image processing and traditional object detection techniques.

Limitations and Challenges:

- Less accurate in complex environments
- Requires manual tuning
- Sensitive to lighting changes

Use in Our Project:

- Used for **video capture and frame processing**
- Converts frames into usable input for YOLOv8
- Handles image preprocessing tasks

2.1.5 Hart, Nilsson, Raphael – A* Algorithm

This paper introduces the A* algorithm for optimal path finding using heuristics.

Limitations and Challenges:

- High computation for large grids
- Not suitable for dynamic environments

Use in Our Project:

- Used for **path planning**
- Computes shortest path between start and goal
- Avoids obstacles detected using YOLOv8

2.1.6 Ultralytics – YOLOv8 Documentation

YOLOv8 is an advanced object detection model with high speed and accuracy.

Limitations and Challenges:

- Requires GPU for real-time inference
- Needs properly annotated datasets

Use in Our Project:

- Used as the **main detection model**
- Provides high accuracy and real-time performance
- Integrated with OpenCV pipeline

2.1.7 Roboflow – Computer Vision Platform

Roboflow provides tools for dataset creation and annotation.

Limitations and Challenges:

- Dataset bias may affect accuracy
- Annotation errors impact results

Use in Our Project:

- Used for **dataset annotation and preprocessing**
- Helps train custom YOLOv8 model
- Improves detection accuracy

2.2 Summary of Research Gaps

- Traditional methods lack accuracy in complex environments
- Earlier systems faced latency issues
- Deep learning requires high computational resources
- Path planning algorithms need adaptation for dynamic environments

Table 2.1: Comparison of Object Detection Frameworks

Parameter	Traditional ML (Haar/HOG)	YOLOv8 (Deep Learning)
Detection Speed	Moderate	Very High (>30 FPS)
Accuracy in Complex Scenes	Low	High
Bounding Box Extraction	No	Yes
Hardware Requirement	Low	High (Laptop/GPU preferred)

Table 2.2: System Parameters and Targets

Component	Parameter	Target Specification
External Camera	Resolution	640 × 480 pixels
External Camera	Frame Rate	30 FPS
Central Laptop	Object Detection Model	YOLOv8 (best.pt)
ESP8266	Communication Protocol	UDP / Wi-Fi
Arduino Uno	Microcontroller Clock	16 MHz

Chapter 3

Work Done Before Mid-Sem

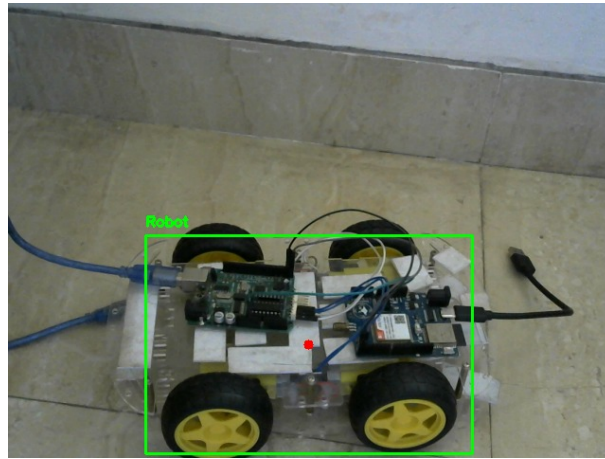


Figure 3.1: Robot Detection

3.1 Object Detection

3.1.1 Theory

Object detection is the process of identifying and locating objects in an image.

Key Components

- Bounding box
- Class label
- Confidence score

3.2 Robot Image Data Collection Using OpenCV

In order to train and test the robot detection system, a dataset of robot images was collected using a USB camera connected to the laptop. The image acquisition process was implemented using the OpenCV library in Python, which provides functions for real-time video capture and image processing.

The camera is initialized using the `cv2.VideoCapture()` function, which allows the system to access the webcam connected to the laptop. The resolution of the captured frames is set to 640×480 pixels to ensure consistent image quality while maintaining efficient processing speed. The camera continuously captures frames from the video stream and displays them on the screen for monitoring.

3.2.1 YOLO Algorithm

YOLO (You Only Look Once) is a real-time object detection model.

- Single-pass detection
- High speed and accuracy
- Suitable for real-time robotics

$$Output = (x, y, w, h, class, confidence)$$

- Implemented YOLO-based obstacle detection
- Developed robot tracking system

3.3 Dataset Preparation and Model Training

After capturing images of the robot using the webcam, the collected images were used to create a dataset for training the object detection model. For this purpose, we used Roboflow, an online platform that helps in managing and preparing datasets for computer vision applications. In Roboflow, a new project named “robot1” was created and configured as an object detection project. All the captured images were uploaded to this project.

Next, the images were manually annotated and labeled. During this process, bounding boxes were drawn around the robot in each image to indicate the exact location of the robot in the frame. This labeling helps the model learn how to identify and detect the robot in different positions and backgrounds. After completing the annotation process, the labeled images were organized into a dataset.

Roboflow was then used to generate the dataset in YOLOv8 format, which is compatible with the YOLO object detection model. The platform provided a code snippet that allowed us to directly download the dataset into a training environment. This code was executed in Google Colab, a cloud-based platform for running Python programs.

Using this dataset, the YOLOv8 model was trained to detect the robot. After the training process was completed, the system generated a trained model file called `best.pt`, which contains the optimized weights used for real-time robot detection.

3.4 Robot Detection and Position Estimation

After training the object detection model and obtaining the best.pt file, a detection program was developed to track the robot in real time using a webcam. The program uses the YOLOv8 model to detect the robot in each frame captured from the camera. When the webcam captures a frame, the trained model processes the image and identifies the robot by drawing a bounding box around it. From this bounding box, the coordinates of the robot are extracted.

The system calculates the center point of the detected robot using the bounding box coordinates. These coordinates represent the robot's current position in the environment. The detected robot location is then displayed on the screen with a bounding box and a center point marker. The position coordinates are also printed in the terminal for further processing.

3.5 System Overview

The system works in the following steps:

1. Capture video input
2. Detect obstacles using YOLO
3. Track robot using tracker

3.6 Sample Object Detection Code

Below is the object detection code using YOLO:

```
1 from ultralytics import YOLO
2 import cv2
3
4 # Load trained model
5 model = YOLO('best_obstacle.pt')
6
7 # Start camera
8 cap = cv2.VideoCapture(0)
9
10 while True:
11     ret, frame = cap.read()
12
13     # Perform detection
14     results = model(frame)
15
16     # Draw bounding boxes
17     for r in results:
18         for box in r.bboxes:
19             x1, y1, x2, y2 = map(int, box.xyxy[0])
20             cv2.rectangle(frame, (x1,y1), (x2,y2), (0,255,0), 2)
21
22     # Display output
23     cv2.imshow("Detection", frame)
24
25     if cv2.waitKey(1) & 0xFF == ord('q'):
26         break
27
28 cap.release()
29 cv2.destroyAllWindows()
```

Listing 3.1: YOLO Object Detection Code

Chapter 4

Work Done After Mid-Sem

After the mid-semester evaluation, the project was further enhanced by integrating wireless communication, improving robot control, transitioning to a dynamic robot detection system, and stabilizing real-time performance.

4.1 Wireless Communication using ESP8266

A major improvement in the system was the introduction of the ESP8266 module for wireless communication. The ESP8266 was configured in station mode and connected to a Wi-Fi network using a static IP address.

UDP communication was implemented to receive control commands from the computer system. These commands include movement instructions such as forward, backward, left, right, and stop. The ESP8266 acts as a bridge between the computer and the robot by forwarding received commands to the Arduino using serial communication.

Reason for use: UDP (User Datagram Protocol) was explicitly chosen over TCP because it is a fast, connectionless protocol with minimal latency. In real-time robotic control, if a packet is delayed, it is better to drop it and process the newest command rather than forcing a delayed packet through, which would cause physical input lag.

4.2 Arduino-Based Robot Control

The Arduino was used to directly control the robot motors through a motor driver shield. It receives commands from the ESP8266 and executes movement operations accordingly.

Different motion functions were implemented such as forward movement, backward movement, left turn, right turn, and stop. Each movement was designed to run for a short duration and then automatically stop, ensuring controlled and precise motion.

Reason for use: This modular architecture separates the "brain" (Python/Computer) from the "muscle" (Arduino). The Arduino strictly handles the low-level PWM signals and hardware timing, freeing up the computer to focus entirely on heavy computational tasks like vision and pathfinding.

4.3 Improved System Integration

The integration between computer vision, wireless communication, and hardware control was significantly improved after mid-sem.

The system now operates in a complete pipeline:

- Detection and navigation decisions are made on the computer
- Commands are transmitted wirelessly via ESP8266
- Arduino executes motor actions based on received commands

Reason for use: Ensures seamless interaction between modules. By clearly defining the boundaries of software and hardware, synchronization issues are minimized, and debugging becomes much easier if a specific part of the pipeline fails.

4.4 System Design and Working

4.4.1 Overall Idea

The developed system integrates computer vision, path planning, and wireless communication to control a robot in real-time.

The system works as follows:

- A camera captures the environment
- The robot is dynamically identified and tracked
- Obstacles are detected using a deep learning model
- A path is generated using A* algorithm
- Commands are sent to the robot via Wi-Fi

Flow:

Camera → Detection → Path Planning → Robot Control

4.4.2 Network and Model Setup

The system uses UDP communication to send commands to the robot through Wi-Fi.

```
1 ESP_IP = "10.90.17.34"
2 UDP_PORT = 5005
3 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Listing 4.1: UDP Communication Setup

The object detection model is loaded using YOLO:

```
1 yolo_model = YOLO('best_obstacle.pt')
```

Listing 4.2: YOLO Model Loading

Reason for use:

- UDP is used for fast, low-latency communication.
- YOLO is loaded upfront so it can detect static and dynamic obstacles in real-time with high inference speed.

4.4.3 Grid System (Environment Mapping)

The camera frame is divided into a grid for navigation.

```
1 GRID_SIZE = 20
```

Reason for use: Pathfinding pixel-by-pixel is computationally heavy and inefficient. The grid system discretizes the continuous environment into distinct nodes, creating the structured 2D weighted graph that algorithms like A* require to function optimally.

4.4.4 Calibration (Homography)

The user selects four points to transform the perspective of the camera view.

```
1 homography_matrix = cv2.getPerspectiveTransform(src_pts, dst_pts)
```

Reason for use: A standard camera captures an angled view with perspective distortion (where distant objects appear smaller). The pathfinding algorithms require a strictly uniform 2D plane. Homography transforms the raw feed into a flat, top-down 2D map, ensuring that distances and angles calculated in pixels perfectly match the real world.

4.5 Transition to Pulse-Based Robot Detection

Prior to the mid-semester evaluation, the system relied on a Convolutional Neural Network (CNN) trained via Roboflow to detect the robot. However, this approach was replaced with a "Pulse-Based" control and detection logic.

In this updated approach, the system sends a randomized sequence of movement and stop commands (wiggling) to the robot. Using OpenCV's background subtraction (`createBackgroundSubtractorMOG2`), the system identifies moving blobs in the frame and scores them based on how perfectly their motion correlates with the transmitted command sequence.

Reason for use (Advantages over CNN):

- **Unique Identification:** If two physically identical robots enter the frame, a CNN cannot differentiate between them. The pulse method uniquely identifies the specific robot responding to network commands.
- **Appearance Invariance:** A CNN requires retraining if the robot gets damaged, painted, or altered. The pulse method tracks synchronized motion, making it entirely invariant to the robot's visual appearance.
- **Scale Invariance:** CNN models can fail if the camera is placed higher or further away than the training data allowed for. Background subtraction identifies pixel variations dynamically, regardless of object scale.

4.6 Robot Detection and Tracking

4.6.1 Motion Detection Scoring

```
1 motion_detector = cv2.createBackgroundSubtractorMOG2()
```

Reason for use: Isolates moving objects in the frame so they can be scored against the transmitted pulse sequence, effectively filtering out random background noise.

4.6.2 Continuous Tracking

```
1 tracker = cv2.TrackerCSRT_create()
```

Reason for use: Once the pulse-based logic positively identifies the robot, the CSRT tracker locks on to provide continuous, high-accuracy bounding box coordinates without needing to constantly stop and wiggle the robot.

4.6.3 Kalman Filter Smoothing

```
1 kf = cv2.KalmanFilter(4, 2)
```

Reason for use: Used for robust robot location estimation. Computer vision tracking inherently contains sensor noise and jitter. The Kalman Filter models the physics of the robot's movement to estimate its true location, keeping the navigation stable even if the camera drops a frame or tracking briefly glitches.

4.7 Robot Direction Detection

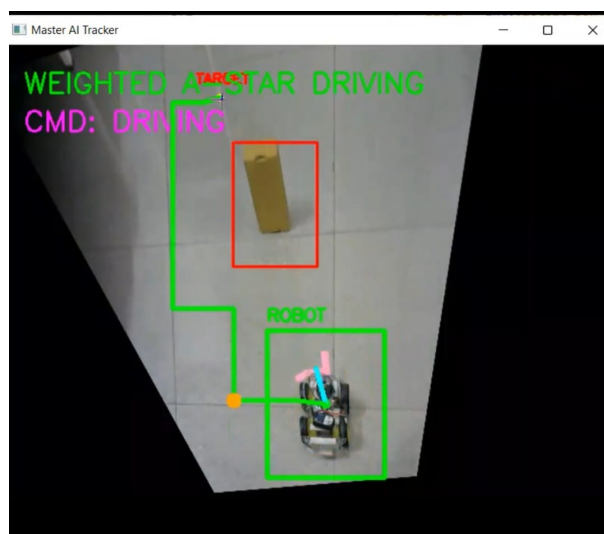


Figure 4.1: Robot Direction


```

1 pink_mask = cv2.inRange(...)
2 current_robot_heading = atan2(...)

```

Reason for use: A standard tracking bounding box only provides a center (X,Y) coordinate, offering no information about which way the robot is facing. The pink mask provides a secondary reference point on the front of the chassis. By calculating the angle between the center of the robot and this marker, the system derives an orientation vector essential for rotational commands.

4.8 Target Selection

User selects a destination point via mouse click:

```

1 target_grid_x, target_grid_y

```

Reason for use: Acts as the dynamic human-in-the-loop input, defining the exact final node for the graph-based pathfinding algorithm.

4.9 A* Path Planning

```

1 def a_star_search(start, goal, obstacles):

```

Features:

- Uses Manhattan distance heuristics.
- Avoids hard obstacles detected by the YOLO model.
- Implements "soft obstacle" danger zones (padding) around objects to prevent clipping.

Equation:

$$f(n) = g(n) + h(n)$$

Reason for use: A* is utilized for optimal path planning. It mathematically guarantees the shortest possible route while balancing computational speed by using a heuristic (Manhattan distance) to effectively guide the search toward the goal, avoiding unnecessary node exploration.

4.10 Obstacle Detection using YOLO

```

1 results = yolo_model(raw_frame)

```

Reason for use: While the robot uses pulse-logic, foreign obstacles (like boxes, hands, or other objects) must be detected dynamically. YOLO is implemented because of its high-speed inference, allowing the system to instantly update the weighted A* graph with new "hard obstacles" if the environment changes.

4.11 Navigation Logic

```
1 if diff > 30:
2     cmd = 'D'
3 elif diff < -30:
4     cmd = 'A'
5 else:
6     cmd = 'W'
```

Reason for use: Compares the calculated path angle with the current physical robot heading. It triggers rotational adjustments (A/D) or forward thrust (W) to ensure the robot stays on the generated path.

4.12 Lookahead Point

```
1 lookahead_distance = 80
```

Reason for use: Instead of navigating strictly from one adjacent grid cell to the next—which would cause the robot to rigidly zigzag and stutter—a pure-pursuit logic is implemented. The `lookahead_distance` forces the robot to target a waypoint further down the path, resulting in smooth, continuous driving curves.

4.13 State Machine

State	Purpose
CALIBRATION	Set perspective homography matrix
WARMUP	Initialize system and await manual start
WIGGLING	Execute pulse commands to detect robot
LOCKING	Verify and lock tracking onto highest scoring blob
AWAITING_TARGET	Standby mode waiting for user destination input
NAVIGATION	Active A* pathing and motor execution

Reason for use: A strict state machine prevents race conditions in the software pipeline. It acts as a safeguard, ensuring that algorithms only execute when their prerequisites are fully met (e.g., preventing the A* algorithm from running before the homography calibration is completed and the robot is locked).

Chapter 5

Conclusion

This project successfully demonstrates the integration of computer vision, path planning, and embedded systems to achieve autonomous robot navigation.

The system is capable of detecting obstacles using a deep learning model and dynamically identifying the target robot using a resilient pulse-based motion tracking technique. By transforming the perspective via homography and converting the environment into a discrete grid representation, highly efficient real-time optimal path planning is achieved using the A* algorithm. The use of Kalman filtering ensures mathematically stable tracking coordinates.

Wireless UDP communication utilizing the ESP8266 enables ultra-low-latency transmission of navigation commands directly to the robot. The modular Arduino-based motor control system executes these instructions effectively, offloading hardware processing to produce accurate physical locomotion.

The entire architecture is governed by a strict state machine, functioning as an automated pipeline that moves seamlessly from environmental perception to physical actuation.

This work highlights the practical engineering applications of object detection, correlation-based tracking, algorithmic optimal pathfinding, and wireless hardware control. It effectively illustrates how decoupled software processing and embedded hardware systems can be harmonized to build intelligent robotic frameworks.

Limitations

Despite the successful implementation, the system has certain limitations. The USB camera (Logitech) used in this project has a limited field of view, which restricts the operational range of the navigation system. As a result, the system performs reliably only in short-distance environments and is not suitable for large-scale or long-distance navigation scenarios.

Additionally, since the system relies solely on vision-based perception, it lacks the ability to accurately distinguish between real physical objects placed on the floor and similar patterns or drawings that resemble obstacles. This can lead to incorrect obstacle detection in certain cases. To overcome this limitation, the integration of additional sensors such as depth sensors, LiDAR, or ultrasonic sensors would be required to provide more reliable environmental understanding.

Chapter 6

Future Work

Although the current system demonstrates successful real-time robot navigation, several improvements can be made to enhance performance, scalability, and robustness.

6.1 PID-Based Motion Control

In the present system, robot movement is controlled using pulse-based commands with fixed delays. This approach, while simple, may lead to inaccuracies such as overshooting or oscillations.

In future work, a PID (Proportional-Integral-Derivative) controller can be implemented to achieve precise motion control. The PID controller can continuously adjust motor speeds based on the error between the desired path and the robot's actual position and orientation. This will result in smoother motion, better stability, and improved accuracy.

6.2 Multiple Camera System

Currently, the system relies on a single camera for environment perception. This limits the field of view and may cause tracking issues when the robot is partially occluded.

A multi-camera setup can be introduced to provide better coverage of the environment. Using multiple cameras, the system can:

- Improve tracking accuracy
- Reduce blind spots
- Enable larger area navigation

Note: This approach is still valid and widely used in advanced robotics and surveillance systems.

6.3 SLAM (Simultaneous Localization and Mapping)

The current system depends on a predefined environment and external camera. A future improvement would be to implement SLAM techniques, allowing the robot to build its own map and localize itself without external vision.

This would make the system fully autonomous and independent of external cameras.

6.4 Sensor Fusion

Currently, the system relies mainly on vision. Integrating additional sensors such as ultrasonic sensors, LiDAR, or IMU can improve reliability.

Sensor fusion can help in:

- Better obstacle detection
- Accurate localization
- Robust performance in dynamic environments

6.5 Dynamic Obstacle Handling

The current system primarily focuses on static obstacles. Future improvements can include handling dynamic obstacles (moving objects), which requires continuous re-planning and faster decision-making.

6.6 Improved Communication Protocols

Although UDP provides fast communication, it does not guarantee packet delivery. Future work can explore:

- Reliable communication protocols
- Error detection and correction
- Adaptive communication strategies

Bibliography

- [1] A. Tselishchev, “Autonomous Navigation of a Mobile Robot Using Overhead Camera and Computer Vision Methods for Time-Critical Tasks,” *IEEE Conference Publication*, Nov. 2024.
- [2] T. Pribanic, H. Dzapu, and M. Cifrek, “Global Vision Based Tracking of Multiple Mobile Robots in Subpixel Precision,” *IEEE Region 8 International Symposium on Video/Image Processing and Multimedia Communications*, 2007.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [5] Ultralytics, “YOLOv8 Documentation,” Available: <https://docs.ultralytics.com/>
- [6] Roboflow, “Roboflow Computer Vision Platform Documentation,” Available: <https://roboflow.com/>
- [7] Espressif Systems, “ESP8266 Wi-Fi Module Datasheet,” Available: <https://www.espressif.com>
- [8] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” 1960. Available: <https://ieeexplore.ieee.org/document/1104570>
- [9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” 1968. Available: <https://ieeexplore.ieee.org/document/4082128>